# Topik 04: Struktur Kontrol Perulangan (Loops) dan Pengindeksan Koleksi Data

## 4.1 Konsep Interval Setengah Terbuka pada Fungsi range()

Dalam bahasa Python, fungsi bawaan range() digunakan untuk menghasilkan deret bilangan bulat terurut. Pemanggilan fungsi dengan format range(start, stop) menerapkan prinsip interval setengah terbuka [start, stop), di mana nilai awal (start) bersifat inklusif, sedangkan nilai akhir (stop) bersifat eksklusif.

Sintaksis Dasar Perulangan for:

```
    for elemen in range(start, stop):
        # blok instruksi yang diulang
```

Contoh Program 4.1: Menghitung Akumulasi Deret

```
1 | n = 4
2 | total = 0
3 | for i in range(1, n):
4 |     total += i
5 | print("Total akumulasi deret:", total)
```

Penjelasan Eksekusi Program 4.1: Pada baris ke-3, range(1, 4) membangkitkan deret angka 1, 2, dan 3. Nilai n (4) TIDAK diikutsertakan. Iterasi pertama: i=1, total=1. Iterasi kedua: i=2, total=3. Iterasi ketiga: i=3, total=6. Ketika i mencapai 4, perulangan berhenti. Program mencetak keluaran: 6.

## 4.2 Pengindeksan Berbasis Nol (Zero-Based Indexing) pada Objek List

Struktur data list pada Python menyimpan sekumpulan nilai terurut di mana indeks elemen pertama SELALU dimulai dari posisi 0. Elemen terakhir pada sebuah list yang memiliki panjang N elemen menempati indeks N - 1.

Contoh Program 4.2: Akses Elemen List dan Kesalahan IndexError

```
1 | data = [10, 20, 30]
2 | print("Panjang list:", len(data))
3 | print("Elemen pertama:", data[0])
4 | print("Elemen terakhir yang valid:", data[len(data) - 1])
```

Perhatian: Mengakses indeks data[len(data)] pada list dengan 3 elemen akan memicu eksepsi runtime IndexError, karena indeks valid hanyalah 0, 1, dan 2.

# Topik 05: Struktur Kontrol Percabangan (Branching)

(Topik 05 ini adalah bagian sintetis tambahan yang disusun untuk prototipe.)

## 5.1 Percabangan if, elif, dan else

Percabangan memungkinkan program memilih blok instruksi sesuai kondisi. Python memeriksa kondisi dari atas ke bawah. Pada rantai if - elif - else, hanya SATU blok yang dijalankan, yaitu blok pertama yang kondisinya benar. Blok elif dan else yang berada di bawahnya dilewati.

Contoh Program 5.1: Rantai Penilaian

```
1 | nilai = 80
2 | if nilai >= 90:
3 |     print("A")
4 | elif nilai >= 75:
5 |     print("B")
6 | else:
7 |     print("C")
```

---

Penjelasan Eksekusi Program 5.1: Kondisi pada baris ke-2 bernilai salah karena 80 kurang dari 90. Python lanjut memeriksa baris ke-4: 80 lebih besar dari 75, sehingga program mencetak B. Blok else pada baris ke-6 dilewati.

Perhatian: Dua pernyataan if yang terpisah BUKAN satu rantai. Setiap if diperiksa sendiri-sendiri, sehingga dua blok bisa sama-sama dijalankan. Penulisan "else if" (dua kata) bukan sintaks Python yang valid. Gunakan elif.

## 5.2 Operator Penugasan (=) dan Operator Perbandingan (==)

Tanda = adalah operator penugasan: menyimpan nilai ke dalam variabel. Tanda == adalah operator perbandingan: memeriksa apakah dua nilai sama dan menghasilkan True atau False, tanpa mengubah variabel apa pun. Hasil perbandingan yang tidak disimpan atau dipakai akan hilang begitu saja.

Contoh Program 5.2: Perbandingan yang Tidak Mengubah Variabel

```
1 | skor = 10
2 | skor == skor + 5
3 | print(skor)
```

---

Penjelasan Eksekusi Program 5.2: Baris ke-2 membandingkan 10 dengan 15 dan menghasilkan False, tetapi hasilnya tidak disimpan. Nilai skor tetap 10, sehingga program mencetak keluaran: 10.

Perhatian: Menulis if x = 5: memicu SyntaxError karena operator = tidak boleh dipakai sebagai kondisi. Untuk membandingkan, tulislah if x == 5:.

## 5.3 Operator Logika and, or, dan not

Operator and bernilai True hanya jika kedua sisi bernilai True. Operator or bernilai True jika salah satu sisi bernilai True. Operator not membalik nilai kebenaran. Kondisi x > 5 or x < 10 selalu benar untuk bilangan apa pun, karena setiap bilangan pasti lebih besar dari 5 atau lebih kecil dari 10.

Contoh Program 5.3: Nilai Operator Logika

```
1 | a = True
2 | b = False
3 | print(a and b)
4 | print(a or b)
5 | print(not a)
```

---

Penjelasan Eksekusi Program 5.3: a and b bernilai False karena b salah. a or b bernilai True karena a benar. not a membalik True menjadi False. Keluaran program berturut-turut: False, True, False.